

Week 11

2. Named Pipes (II): FIFO Examples

Dr. Andreas S. Maniatis

Assistant Professor

School of Computer Science

COMP.2560 – System Programming [2025F]

Monday, November 12th, 2025



University of Windsor

Content

Review:

- man 7 pipe
- man 7 fifo

// mknod fifoname p

- man 3 mkfifo

Example 6: Client / Server

Example 7: sendmessage / rcvmessage

Example 8: 2-way CS communication

Example 9: Multi-Client CS

Part I

Named Pipes (Review)



Named pipes

FIFOs or Named Pipes

FIFOs (First In First Out), also called *Named Pipes*, offer the following advantages over pipes:

- They have a name that exists in the **file system**.
They can be used by **unrelated processes**.
- They exist until explicitly **deleted**.

The `mkfifo()` library function

`mkfifo()`

Synopsis:

```
int mkfifo(const char *path, mode_t mode)
```

`mkfifo()` returns 0 if OK, -1 otherwise.

Creating a FIFO is similar to creating a file.

Example: `mkfifo("/tmp/channel.fif", 0755);`

Note

`mkfifo()` is actually a library function (`#include <sys/stat.h>`) that invokes the system call `mknod()`, used to create special files.

FIFO manipulation

Once a FIFO has been created, it can be treated as a file. In particular, the system calls `open()`, `close()`, `read()`, `write()` and `unlink()` (to delete a file) can be used on a FIFO.

- To **create** a FIFO: Use `mkfifo` library function.
- To **remove** a FIFO: Use `unlink` system call:
`unlink(fifoname)`
- To **read from a FIFO**: Use `open(fifoname, O_RDONLY)`, then use `read` system call.
- To **write to a FIFO**: Use `open(fifoname, O_WRONLY)`, then use `write` system call.

FIFO manipulation

By default, we have:

- Calling `open()` for read-only blocks the caller until some other process opens the FIFO for writing.
- Calling `open()` for write-only blocks the caller until some other process opens the FIFO for reading.
- If a process writes to a FIFO that no process has opened for reading, the signal **SIGPIPE** will be generated.
- When the last writer for a FIFO closes the FIFO, an **end-of-file** will be generated for the reader.
- Like pipes, FIFOs are one-way communication channels.

Note

In case of multiple processes writing to the same pipe, a `write` of up to `PIPE_BUF` bytes is guaranteed to be atomic.

Part II

Named Pipes (Examples)



○ Example 6: Client/Server



Example 6: A client/server application

The goal of the following example (Example 6) is to write a client/server application where a server accepts data from clients using the FIFO `~/myserver`.

○ server1.c



Example 6: A client/server application...(server part)

```
#include <wait.h>
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/stat.h>
#include <stdlib.h>
#include <strings.h>
#include <string.h>

void child(pid_t client);

int main(int argc, char *argv[]){
    int fd, status;
    char ch;
    pid_t pid;

    unlink("/tmp/server");

    if(mkfifo("/tmp/server", 0777)){
        perror("main");
        exit(1);
    }

    while(1){
        fprintf(stderr, "Waiting for a client\n");

        fd = open("/tmp/server", O_RDONLY);
        fprintf(stderr, "Got a client: ");
        read(fd, &pid, sizeof(pid_t));
        close(fd);

        fprintf(stderr, "%ld\n", pid);

        if(fork()==0)
            child(pid);
        else {
            waitpid(0, &status, WNOHANG);
            sleep(3);
            //printf("child pid=%d terminated.\n", wait(NULL));
        }
    }
}
```

Example 5: A client/server application...(server part)

```
void child(pid_t pid){
    char fifoName[100];
    char newline = '\n';
    int fd, i;

    sprintf(fifoName, "/tmp/fifo%d", pid);

    unlink(fifoName);

    mkfifo(fifoName, 0777);

    fd = open(fifoName, O_WRONLY);

    for(i=0; i < 10; i++){
        write(fd, fifoName, strlen(fifoName));
        write(fd, &newline, 1);
        sleep(1);
    }
    close(fd);
    unlink(fifoName);
    exit(0);
}
```

○ client.c



Example 5: A client/server application...(client.c)

```
//client.c
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/stat.h>
#include <stdlib.h>

int main(int argc, char *argv[]){ // Client

    char ch, fifoName[100];
    int fd;
    pid_t pid;

    while((fd=open("/tmp/server", O_WRONLY))===-1){
        fprintf(stderr, "trying to connect\n");
        sleep(1);
    }

    pid = getpid();
    write(fd, &pid, sizeof(pid_t));

    close(fd);

    sprintf(fifoName, "/tmp/fifo%d", pid);
    sleep(1);

    fd = open(fifoName, O_RDONLY);

    int ret=read(fd, &ch, 1);
    while(ret == 1) {
        fprintf(stderr, "%c", ch);
        ret=read(fd, &ch, 1);
    }
    printf("ret=%d\n", ret);
    close(fd);
}
```

○ Example 7: Send – Receive [sendmessage/rcvmessage]



Content

Example 7 sendmessage/rcvmessage

Note : open FIFO for RD/WR , or BOTH (in rcvmessage.c) , why?

Review textbook: Chapter: 15.5 (pages 552 – 556)

○ rcvmessage.c



Example 7: A client/server application...(rcvmessage)

```

#include <fcntl.h>
#include <stdio.h>
#include <errno.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/stat.h>

#define MSGSIZE 63

char *fifo = "fifo";

int main(int argc, char **argv){
    int fd;
    char msgbuf[MSGSIZE + 1];

    unlink(fifo);

    if (mkfifo(fifo, 0666) == -1){
        if (errno != EEXIST){
            perror("receiver: mkfifo");
            exit(4);
        }
    }

    if ((fd = open(fifo, O_RDWR)) < 0){ //note O_RDWR, change it to O_RDONLY?
        perror("fifo open failed");
        exit(5);
    }

    for (;;){
        int num=read(fd, msgbuf, MSGSIZE + 1);
        if (num < 0){
            perror("message read failed");
            exit(6);
        }
        //including the following when O_RDONLY
        /*
        if (num == 0){
            printf("nothing to read or no write end.\n");
            continue;
        } */
        printf("message received:%s\n", msgbuf);
    }
}

```

○ sendmessage.c



Example 7: A client/server application...(sendmessage)

```

#include <fcntl.h>
#include <stdio.h>
#include <errno.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

//sendmessage "message 1 text" "messgae 2 text" ...

#define MSGSIZE 63

char *fifo = "fifo";

int main(int argc, char **argv) {
    int fd, j, nwrite;
    char msgbuf[MSGSIZE + 1];

    if (argc < 2)
    {
        fprintf(stderr, "usage: send message1 message 2 ..... \n");
        exit(1);
    }

    if ((fd = open(fifo, O_WRONLY|O_NONBLOCK)) < 0) { //if delete O-NONBLOCK
        perror("fifo open failed");
        exit(4);
    }

    for (j = 1; j < argc; j++){
        if (strlen(argv[j]) > MSGSIZE) {
            fprintf(stderr, "message too long %s\n", argv[j]);
            continue;
        }

        strcpy(msgbuf, argv[j]);

        if ((nwrite = write(fd, msgbuf, MSGSIZE + 1)) == -1) {
            perror("message wirte failed");
            exit(5);
        }
    }
    exit(0);
}

```



Example 8: 2-way CS Communication



○ 2-way_CS-Server.c



Example 8: A CS application...(2-way_CS-Server.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <string.h>

#define FIFO_IN "/tmp/server_in"
#define FIFO_OUT "/tmp/server_out"

int main() {
    int fd_in, fd_out;
    char buf[256];
    ssize_t n;

    // Remove any leftover FIFOs
    unlink(FIFO_IN);
    unlink(FIFO_OUT);

    // Create the two named pipes
    if (mkfifo(FIFO_IN, 0666) == -1 || mkfifo(FIFO_OUT, 0666) == -1) {
        perror("mkfifo");
        exit(1);
    }

    printf("Server: waiting for client to connect...\n");

    // Open both FIFOs (read from client, write to client)
    fd_in = open(FIFO_IN, O_RDONLY);
    fd_out = open(FIFO_OUT, O_WRONLY);

    printf("Server: client connected!\n");

    // Communication loop
    while ((n = read(fd_in, buf, sizeof(buf) - 1)) > 0) {
        buf[n] = '\0'; // null-terminate received text
        printf("Client says: %s", buf);

        // Form a simple response
        char reply[256];
        snprintf(reply, sizeof(reply), "Server received: %s", buf);
        write(fd_out, reply, strlen(reply));
    }

    printf("Server: client disconnected.\n");
    close(fd_in);
    close(fd_out);
    unlink(FIFO_IN);
    unlink(FIFO_OUT);
    return 0;
}
```


○ 2-way_CS-Client.c



Example 8: A CS application...(2-way_CS-Client.c)

```
//2-way_CS-Client.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <string.h>

#define FIFO_IN  "/tmp/server_in"
#define FIFO_OUT "/tmp/server_out"

int main() {
    int fd_in, fd_out;
    char buf[256];

    printf("Client: waiting to connect...\n");

    // Wait until the FIFOs exist (created by server)
    while (access(FIFO_IN, F_OK) == -1 || access(FIFO_OUT, F_OK) == -1) {
        printf("Client: waiting for server...\n");
        sleep(1);
    }

    // Open both FIFOs (write to server, read from server)
    fd_in = open(FIFO_IN, O_WRONLY);
    fd_out = open(FIFO_OUT, O_RDONLY);

    printf("Client: connected! Type messages (Ctrl+D to quit)\n");

    // Main loop
    while (fgetc(buf, sizeof(buf), stdin) != NULL) {
        write(fd_in, buf, strlen(buf)); // send to server
        int n = read(fd_out, buf, sizeof(buf) - 1); // get reply
        if (n > 0) {
            buf[n] = '\0';
            printf("Server replied: %s", buf);
        }
    }

    close(fd_in);
    close(fd_out);
    return 0;
}
```

○ Example 9: Multi-Client CS



Example 9

The goal of the following example (example 9) is to write a client/server application where a server creates a child for each client. Then, the child creates a separate FIFO to send data to the client.

○ Multi-Client_CS-Server.c



Example 9: A CS application...(Multi-Client_CS-Server.c)

```
//Multi-Client_CS-Server
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <string.h>
#include <sys/wait.h>

#define SERVER_FIFO "/tmp/server_requests"

void handle_client(pid_t pid);

int main(void) {
    int server_fd;
    pid_t client_pid;

    unlink(SERVER_FIFO);
    if (mkfifo(SERVER_FIFO, 0666) == -1) {
        perror("mkfifo server");
        exit(1);
    }

    printf("Server: waiting for clients...\n");
    server_fd = open(SERVER_FIFO, O_RDONLY);

    while (1) {
        if (read(server_fd, &client_pid, sizeof(pid_t)) > 0) {
            printf("Server: got connection request from client %d\n", client_pid);
            pid_t child = fork();
            if (child == 0) {
                handle_client(client_pid);
                exit(0);
            }
        }
    }
}

// Handle one client in a separate process
void handle_client(pid_t pid) {
    char client_fifo[64];
    sprintf(client_fifo, "/tmp/client_%d", pid);

    int fd = open(client_fifo, O_WRONLY);
    if (fd == -1) {
        perror("open client_fifo");
    }
}
```

○ Multi-Client_CS-Client.c



Example 9: A CS application...(Multi-Client_CS-Client.c)

```
//Multi-Client_CS-Client
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <string.h>

#define SERVER_FIFO "/tmp/server_requests"

int main(void) {
    pid_t pid = getpid();
    char client_fifo[64];
    sprintf(client_fifo, "/tmp/client_%d", pid);

    unlink(client_fifo);
    if (mkfifo(client_fifo, 0666) == -1) {
        perror("mkfifo client");
        exit(1);
    }

    int server_fd = open(SERVER_FIFO, O_WRONLY);
    if (server_fd == -1) {
        perror("open server fifo");
        exit(1);
    }

    // Send our PID to the server
    write(server_fd, &pid, sizeof(pid_t));
    close(server_fd);

    printf("Client %d: waiting for server messages...\n", pid);

    int fd = open(client_fifo, O_RDONLY);
    char buf[256];
    ssize_t n;
    while ((n = read(fd, buf, sizeof(buf) - 1)) > 0) {
        buf[n] = '\0';
        printf("%s", buf);
    }

    close(fd);
    unlink(client_fifo);
    printf("Client %d: done.\n", pid);
    return 0;
}
```


Part III

Attendance Quiz (Not graded)



Quiz



Attendance Quiz

701968234901768023405879312016
670980321768767



774862586034187069210783468019
701968234901768023405879312016
670980321768767

WPOYFXI LFJXI G I E
H I MPHLOM ZFOLI PG
BXNYSXWHYGI JLG I L
OLI EPC OOUJPL Q DZC
NYDWIGL OUPJL Q DZC

Thank you!
Questions?

DEFYXHQGRUOSGRP RN	3236343972474116
QJRMVQJXVRI DKUEVP	893833239594578
KGWBGQFHP RSUP RU	0310019701552750
XURNYUXZ DCSYUUM I	7875465400196596
WHMFSP UIQYSUJYRSHRG	5019353606937419

00110011100111
11111010000100
11000000001111
00111100111100
00101111001100
00000111011011
11001100110011

Dr. Andreas S. Maniatis

Assistant Professor

Andreas.Maniatis@UWindsor.ca

<http://www.linkedin.com/in/andreasmaniatis>



University of Windsor